2.1 Deterministic and Non-deterministic Finite Automata

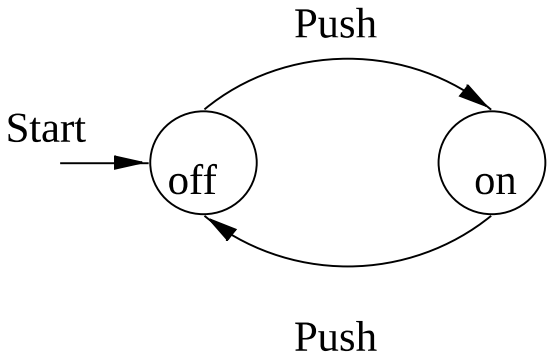
- Automata = abstract computing devices.
- Turing studied Turing Machines (= computers) before there were any real computers
- We will also look at simpler devices than Turing machines (Finite State Automata, Pushdown Automata,...), and specification means, such as grammars and regular expressions.
- NP-hardness = what cannot be efficiently computed.
- Undecidability = what cannot be computed at all.

2.1.1 Finite Automata

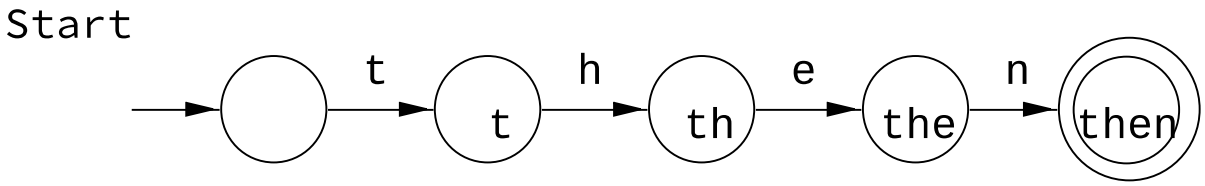
Finite Automata are used as a model for

- Software for designing digital circuits
- Lexical analyzer of a compiler
- Searching for keywords in a file or on the web.
- Software for verifying finite state systems, such as communication protocols.

- Example: Finite Automaton modeling an *on/off* switch



- Example: Finite Automaton recognizing the string *then*



Finite Automata Informally

Protocol for e-commerce using e-money.

Allowed events:

1. The customer can *pay* the store (=send the money-file to the store)

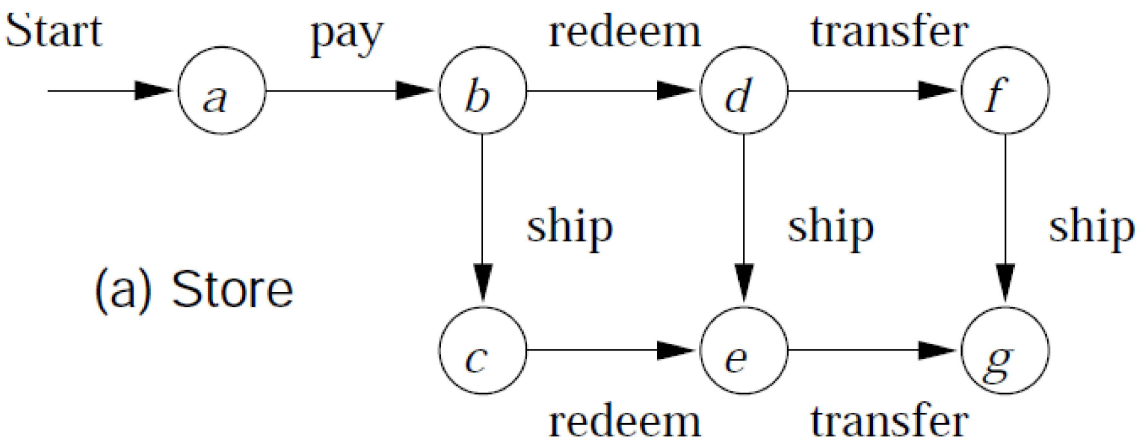
2. The customer can *cancel* the money (like putting a stop on a check)

3. The store can *ship* the goods to the customer

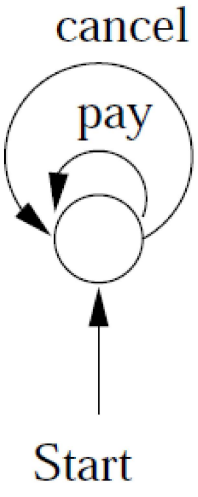
4. The store can *redeem* the money (=cash the check)

5. The bank can *transfer* the money to the store

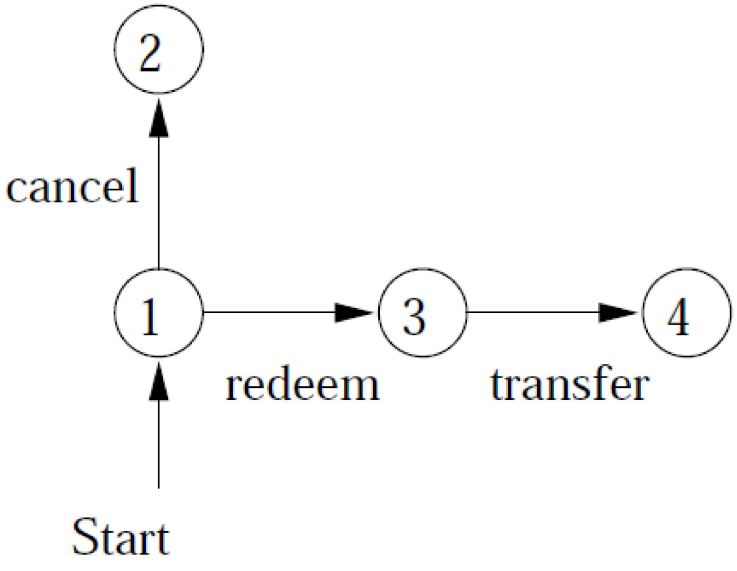
The protocol for each participant:



(a) Store

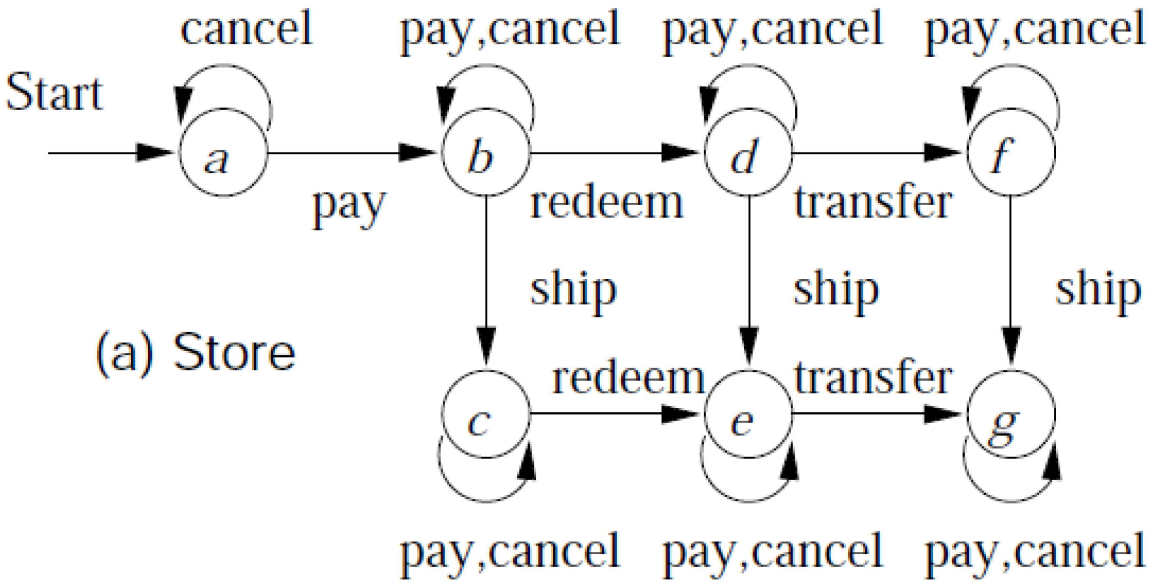


(b) Customer

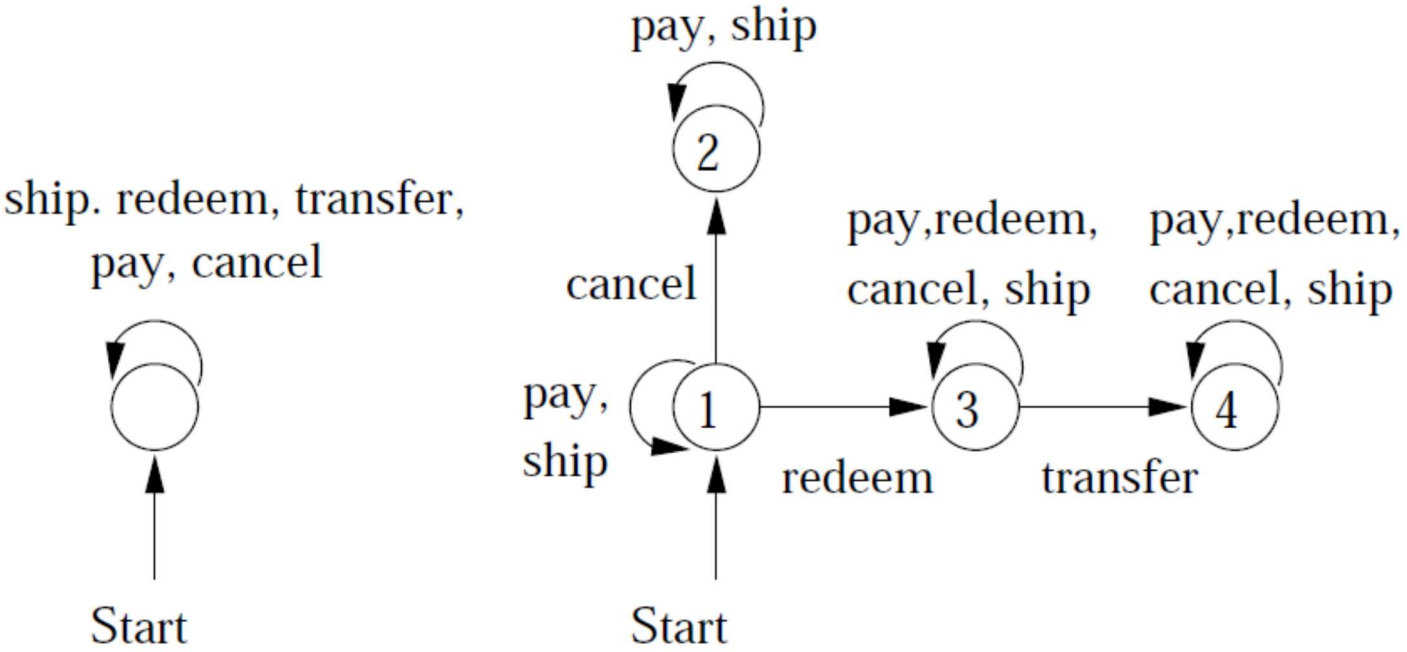


(c) Bank

Completed protocols:



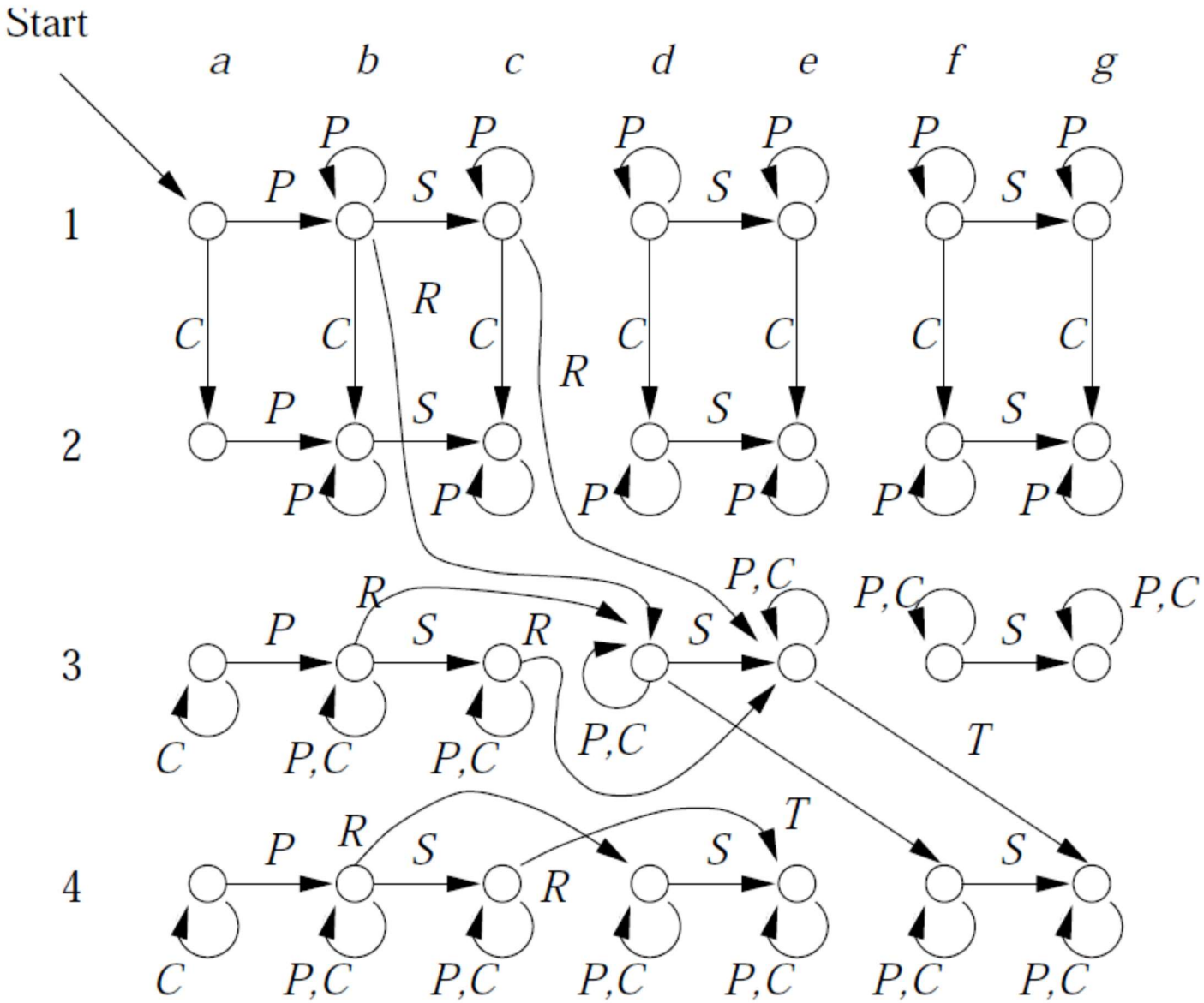
(a) Store



(b) Customer

(c) Bank

The entire automation system:



Deterministic Finite Automata

A Finite Automata can be represented as a five tuple (quintuples):

$M = (Q, \Sigma, \delta, q_0, F)$ Where,

Q : Set of states e.g. : $\{S,A\}$

Σ : Set of Input Valid String
e.g. : $\{d\}$

δ : Set of Transition Function
(Transition over Q $\times$ Σ to Q)
(q,a) → p

$q_0 \in Q$: Start State e.g. : S

F : Set of Final States
e.g.: $\{A\}$ *and* $F \subseteq Q$.

Example: An automaton ***A*** that accepts

$$L = \{x01y : x,y \in \{0,1\}^*\}$$

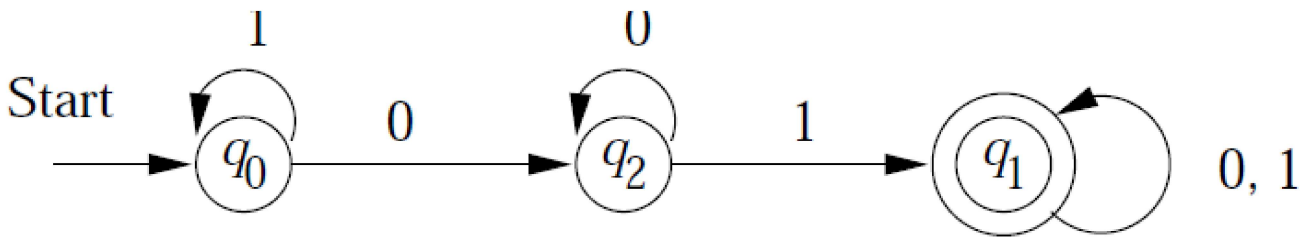
The automaton

$$A = (\{q_0,q_1,q_2\},\{0,1\},\delta,q_0,\{q_1\})$$

as a transition table:

δ	0	1
$\rightarrow q_0$	q_2	q_0
$\star q_1$	q_1	q_1
q_2	q_2	q_1

The automaton ***A*** as a transition diagram:



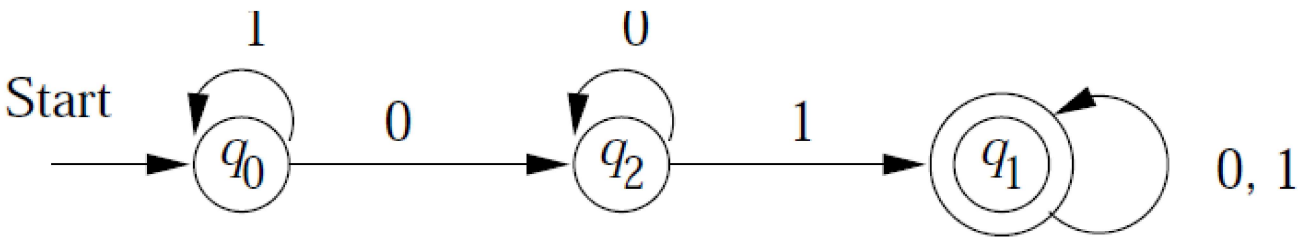
An FA accepts a string

$$w = a_1 a_2 \ldots a_n$$

if there is a path in the transition diagram that

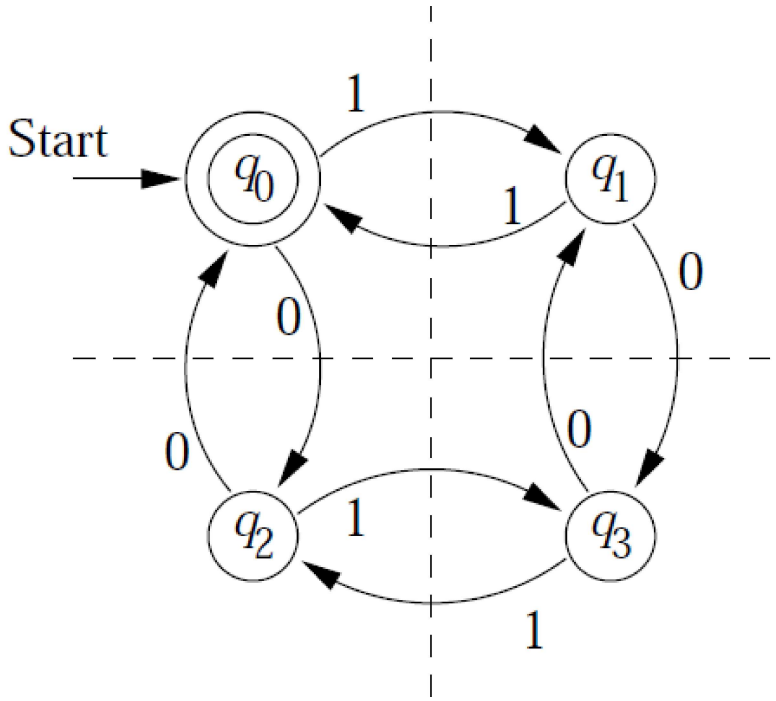
1. Begins at a start state
2. Ends at an accepting state
3. Has sequence of labels $a_1 a_2 \ldots a_n$.

Example: The FA



accepts e.g. the string 1100101

Example:DFA accepting all and only strings with an even number of 0's and an even number of 1's.



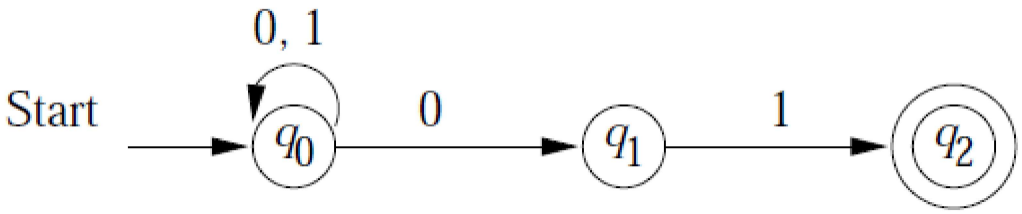
Tabular representation of the Automaton:

δ	0	1
$\star \rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

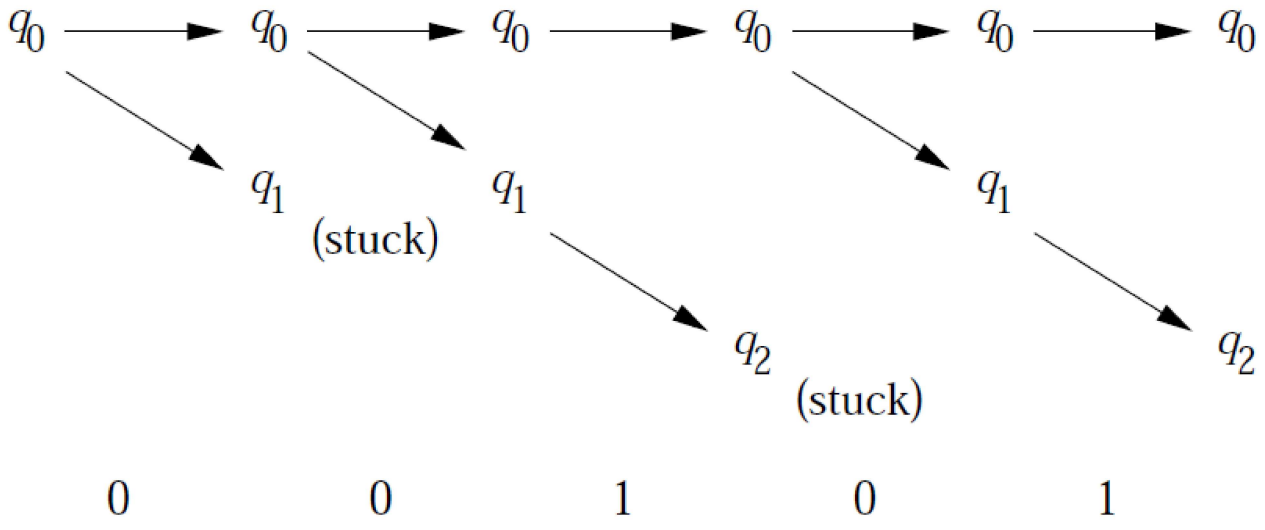
Nondeterministic Finite Automata

A NFA can be in several states at once, or, viewed another way, it can "guess" which state to go to next.

Example: An automaton that accepts all and only strings ending in 01.



Here is what happens when the NFA processes the input 00101.



Formally, a NFA is a quintuples

$$A = (Q,\Sigma,\delta,q0,F)$$

- Q is a finite set of states
- Σ is a finite alphabet
- δ is a transition function from $Q \times \Sigma$ to the powerset of Q
- $q0 \in Q$ is the start state
- $F \subseteq Q$ is a set of final states

Example: The NFA from the previous slide is

$$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$$

where δ is the transition function

δ	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$\star q_2$	$\emptyset$	$\emptyset$

2.1.1 Equivalence of DFA and NFA

- NFA's are usually easier to "program" in.
- Surprisingly, for any NFA N there is a DFA D, such that L(D) = L(N), and vice versa.
- This involves the subset construction, an important example how an automaton B can be generically constructed from another automaton A.
- Given an NFA

$$N = (Q_N, \Sigma, \delta_N, q_0, F_N)$$

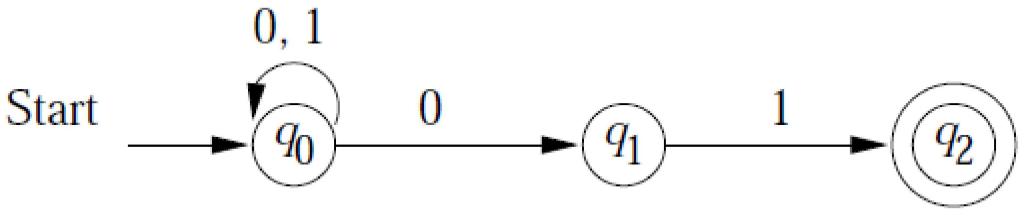
we will construct a DFA such that,

$$D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D).$$

Construction of DFA from NFA

- Sub-Sets construction method

Example: Consider an NFA from page 16, that accepts all the strings ended by 01.



Complete Sub-Sets:

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\star\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

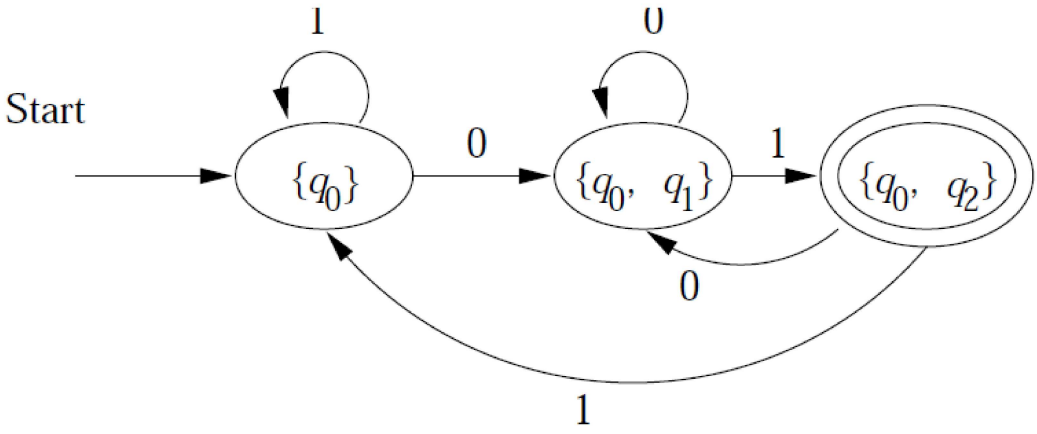
Formulation:

Inputs = 2, States = 3 and
Number of Sub-Sets, $2^3 = 8$.

We can often avoid the exponential blow-up by constructing the transition table for D only for accessible states S as follows:

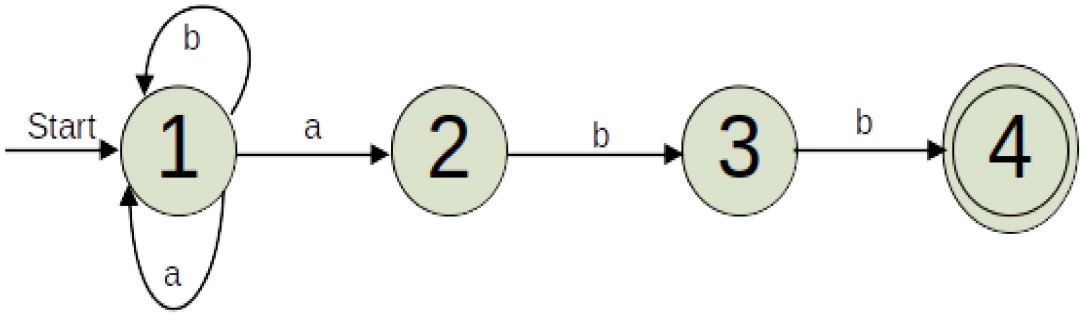
Example: The Sub-Sets DFA with accessible states only.

	0	1
→ $\{q_0\}$	$\{q_0,q_1\}$	$\{q_0\}$
$\{q_0,q_1\}$	$\{q_0,q_1\}$	$\{q_0,q_2\}$
*$\{q_0,q_2\}$	$\{q_0,q_1\}$	$\{q_0\}$



Task:

Construct DFA from the given NFA which accepts $(a|b)^*abb$.



FA with Epsilon - Transition

Informally:

It is the extension of finite automata; with a transition on ε, the empty string, an NFA is allowed to make a transition spontaneously, without receiving an input symbol.

Formally:

An -NFA is a quintuple

$$(Q,\Sigma,\delta,q_0,F)$$

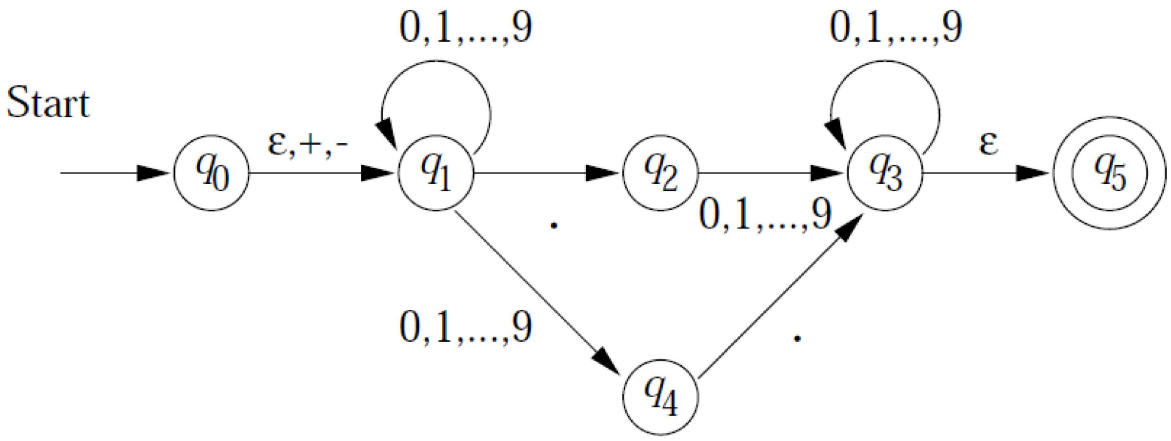
where δ is a function from $Q \times \Sigma \cup \{\epsilon\}$ to the powerset of Q.

Example:

The - NFA that accepts decimal numbers consisting of:

1. An optional + or - sign
2. A string of digits
3. a decimal point
4. another string of digits

One of the strings (2) are (4) are optional



i.e.

```
E = (
        {q0,q1,...,q5},
        {.,+,-,0,1,...,9},
        δ,
        q0,
        {q5}
    )
```

where the transition table for δ is

	ϵ	+,-	.	0,...,9
$\rightarrow q_0$	$\{q_1\}$	$\{q_1\}$	$\emptyset$	$\emptyset$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$	$\{q_1, q_4\}$
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_3	$\{q_5\}$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_4	$\emptyset$	$\emptyset$	$\{q_3\}$	$\emptyset$
$\star q_5$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

ECLOSE

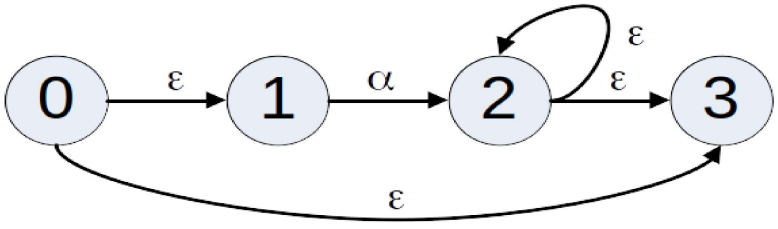
We close a state by adding all states reachable by a sequence $\epsilon\epsilon\cdots\epsilon$.

Algorithm for finding ε-Closure(s):

→ δ is added to ε-closure(s)

→ If *f* is in ε-closure(s), and there is an edge labeled ε from *f* to *u*, then *u* is also added to ε-closure(s). This step is repeated till not state is left which can be added to ε-closure(s).

Workout: Find ε-closure(0) for the following NFA.



Solution: ε-closure(0) = {0, 1, 2}

Sub-Sets construction Algorithm:

1. A = ε-closure(0),
   i.e. start state.

2. For each input symbol a do.
   Let T be set of states to which there is a transition a from states in A.

(Look for the states in A on which there is a transition on input symbol, say a and b.

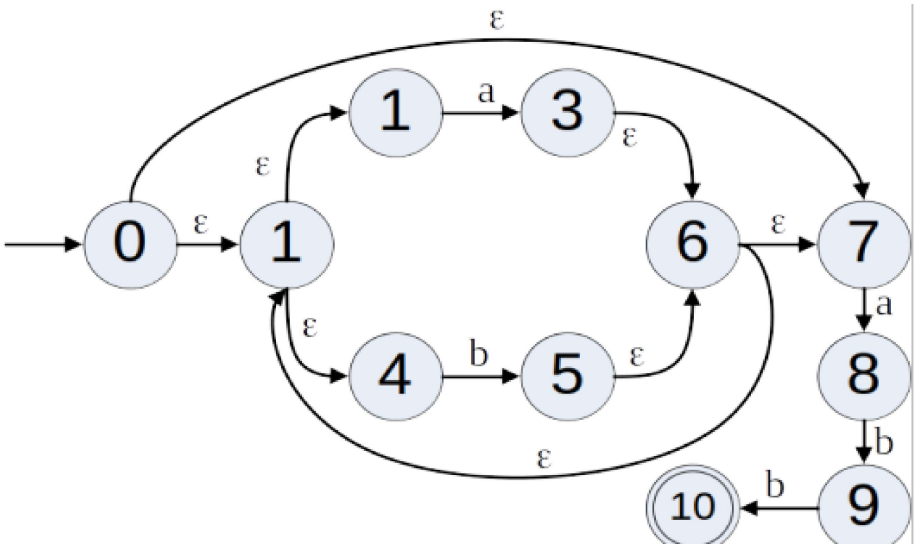
Then,
   T_a = {set of states to which transition made on a}.

   T_b = {set of states to which transition is made on b})
   i.e.
   B = ε-closure(Ta) and
   C = ε-closure(T_b).

Mark A as its transition state has been found.

3. Repeat the step 2 with newly found states i.e. B and C and this process will be repeated till all the newly found states are marked and there is at least one state which contains final state of NFA.

Example: Find the DFA for following ε-NFA.



Where, $\Sigma = \{a, b\}$

Solution:

Applying the *algorithm*,

Let A = ε-closure(0) = {0, 1, 2, 4, 7}

Now, There exists transitions with input symbol a, b on 2,4,7 to 3,5,8.

Thus, $T_a = \{3, 8\}$

$T_b = \{5\}$

Hence, B = ε-closure(T_a)

C = ε-closure(T_b)

Thus, we have found transition for *A* i.e. to *B* on *a*, and to *C* on *b*.

Now, Repeat the process for B,

$$
\begin{aligned}
B &= \varepsilon\text{-closure}(T_a) \\
&= \varepsilon\text{-closure}\{3,8\} \\
&= \varepsilon\text{-closure}(3) \cup \varepsilon\text{-closure}(8) \\
&= \{1, 2, 3, 4, 6, 7\} \cup \{8\} \\
&= \{1, 2, 3, 4, 6, 7, 8\}
\end{aligned}
$$

$$
\begin{aligned}
C &= \varepsilon\text{-closure}(T_b) \\
&= \varepsilon\text{-closure}\{5\} \\
&= \{1, 2, 4, 5, 6, 7\}
\end{aligned}
$$

Let *D'* be the set of marked states.

Thus add *A* to *D'* i.e. *D'* = *{A}*

<u>For B,</u>

$$
\begin{aligned}
B &= \{1, 2, 3, 4, 6, 7, 8\} \\
T_a &= \{3, 8\} \\
T_b &= \{5, 9\} \\
\varepsilon\text{-closure}(T_a) &= \varepsilon\text{-closure}\{3, 8\} = B \\
\varepsilon\text{-closure}(T_b) &= \varepsilon\text{-closure}\{5, 9\} \\
&= \{1,2,4,5,6,7,9\} \\
&= D
\end{aligned}
$$

Thus, we have found transition for B to B on a and B to D on b.
Hence, D' = {A, B, …} i.e. B is marked.

For C,

C = {1, 2, 4, 5, 6, 7}
T_a = {3, 8} (from states 2, 7 on a)
T_b = {5} (from state 4 on b)
ε-closure(T_a) = ε-closure{3, 8} = B
ε-closure(T_b) = ε-closure{5} = C

Thus, transition from C to B on a and C to C on b have found. C is marked.
Hence, D' = {A, B, C, …}

For D,

D = {1, 2, 4, 5, 6, 7, 9}
T_a = {3, 8}
T_b = {5, 10}
ε-closure(T_a) = ε-closure{3, 8} = B
ε-closure(T_b) = ε-closure{5, 10}
= {1,2,4,5,6,7,10}
= E

Thus, we have found transitions for D to B on a and D to E on b. Hence, D is included in marked state,
i.e. D' = {A, B, C, D, …}

For E,

E = {1, 2, 4, 5, 6, 7, 10}
T_a = {3, 8}
T_b = {5}
ε-closure(T_a) = ε-closure{3, 8} = B
ε-closure(T_b) = ε-closure{5} = C

Thus, we have found transitions for E to B on a and E to C on b. Hence, E is included in marked state,
i.e. D' = {A, B, C, D, E}

Now there is no new state and all are marked and also, E is final state as it contains the final state 10.

Final Transition table:

δ	a	b
$\rightarrow A$	B	C
B	B	D
C	B	C
D	B	E
$*E$	B	C

Σ = {a, b},
Q = {A, B, C, D, E},
q0 = A,
F = {E} and
$\boldsymbol{\delta}$ ← shown above.

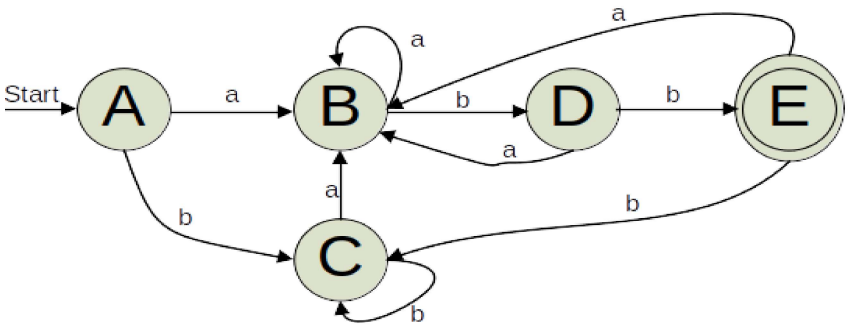


Fig. DFA obtained from above ε-NFA.

Minimization of states:

1. Separate the states into group of halt and non-halt states.

2. Take any two states in a two sup-groups and test whether they are same with respect to the transition function. If so, merge them into one, otherwise separate them into different sub-groups. Then, draw the final TD.

For example: DFA that accepts $(a|b)^*abb$

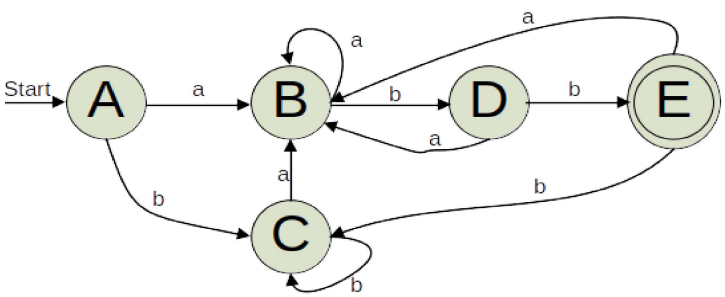


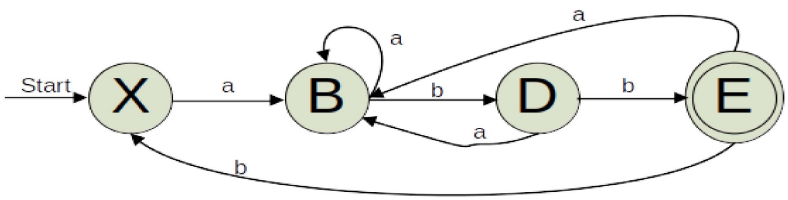
Fig. DFA obtained from above ε-NFA.

Minimization: Looking at the Transition Table.

δ	a	b
$\rightarrow A$	B	C
B	B	D
C	B	C
D	B	E
$*E$	B	C

1. Separation: {A,B,C,D}, {E}
2. {A,C},{B,D},{E}
3. {X}, {B},{D}

Final DFA :



Removal of Inaccessible States:

Algorithm :

1. Mark the start state.

2. Starting from a marked state, mark all the stares for which there exists a transition.

3. Repeat 2 until no more new marked states are added.

4. Remove all the unmarked state as they are inaccessible states.

Removal of Empty-Cycles:

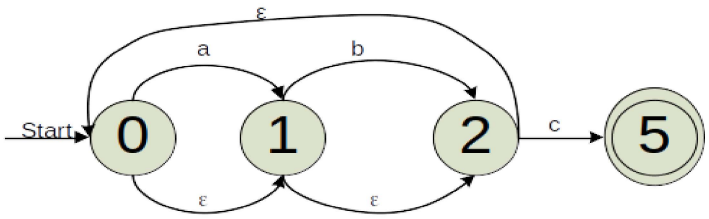
An empty cycles is detected if the system can reach the same state following a set of empty moves only.

Algorithm : Detection & Removal of empty cycles.

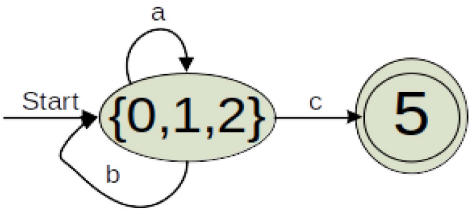
(a) Mark all the states.
(b) Construct a tree with any marked states as root node.
(c) A child node to a parent node is that node for which there exists an empty transition from the parent node.
(d) If the root node reappears, an empty cycle is detected and the path form the root up to the leaf node denotes the empty cycle.
(e)
- Merge all the nodes in the path with the root node.
- Unmark the root state(node).

(f) Repeat steps (a) to (e) until all the states are unmarked.

e.g. :

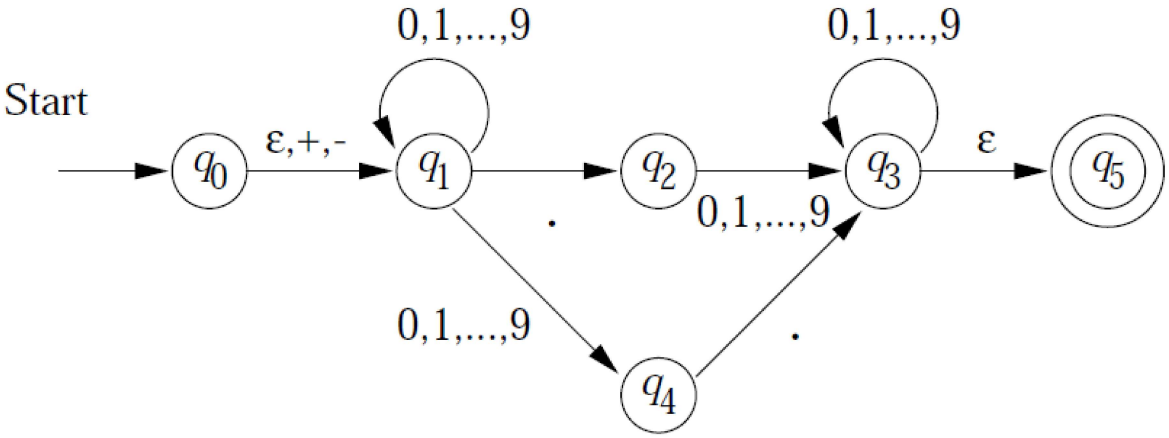


Applying the *Algorithm* :



Assignment:

Construct equivalent DFA from the below NFA.



Machine Equivalence:

Two FSMs M and M' are said to be equivalent; if every string recognized by M is also recognized by M'.

Isomorphic FSMs:

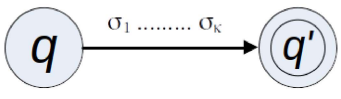
Two FSMs M and M' are said to be isomorphic if M can be obtained by relabeling M' and vice-versa.

2.2.2 Theorem :

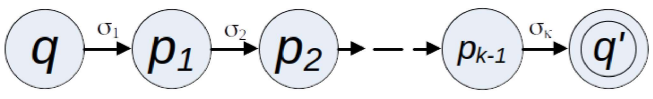
For each NFA, there exists is an equivalent DFA.

Proof :

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a NFA. In order to transform M into an equivalent DFA, various possibilities must be eliminated: transitions $(q, u, q') \in \delta$ such that $u = \varepsilon$ of such that $|u| > 1$; transitions that are missing; and multiple transitions that may be applicable to the same configuration. It is relatively easy to eliminate transitions (q, u, q') with $|u| > 1$. In essence, we introduce new states such as those in figure (b) to replace an arrow in the state diagram such as that shown in figure (a).



(a)



(b)

Formally,

if (q, $\sigma_1 \ldots \sigma_k$, q') is a transition of *M* and $\sigma_1 \ldots \sigma_k \in \Sigma$, k >= 2, then add new (non-halt) states $p_1, \ldots p_{k-1}$ to K and new transitions (q, σ_1, p_1), (p_1, σ_2, p_2), . . ., (p_{k-1}, σ_k, q') to δ .

Let M' = (Q', Σ, δ', q_0', F') be the NFA that results from M when this transformation is carried out for each transition (q, u, q') of M such that |u| > 1.

It should be obvious that M' and M are equivalent, and that |u| <= 1 for each transition (q, u, q') of M'.

2.2.1 Regular Expression

- The language accepted by Finite Automata is easily described by simple expression called Regular Expression.

- A language denoted by a regular expression is called as a Regular Set or Regular Language.

- A Regular Expression uses three operators i.e. Regular Operators,

=> Alternation [|] :
either *a* or *b* (Union +)
=> Concatenation [.] :
a and *b*
=> Closure [{}, *] :
Zero or more numbers of occurrence.

All the following are regular expressions: ε, 0, 1, (0|1), $(1|0)^*$, 00(1|0)*10

The string $(1|0)^*$ represents empty string or any number of 0's and 1's and in any number of occurrences.

Example:

1. $(a|bb|c)a^*$, i.e., aa^*, bba^*, ca^* means either an "a" or a "bb" or a "c" is followed by zero or more a's.

2. $((a|b)(c|d))^*$ indicate either a null string or (ac)*, $(ad)^*$, $(bc)^*$ or $(bd)^*$ i.e., zero or more of ac, ad, bc or bd concatenated i.e., acad, adbc, adbdbd etc.

3. $((a|b)5)^*$ would give string whose lengths are multiples of 5. This can be further modified to more strings whose length mod 5 is a constant.

CW: Find the regular expression that generates a string of 0's and 1's such that the string read as a binary number is divisible by 5.

2.3 Properties of Regular expressions

1. Any terminal symbol in Σ, including ε and ϕ are regular expressions.

2. Union of two regular expressions R_1 and R_2, written R_1+R_2 $(R_1|R_2)$, is also regular expression.

3. The concatenation of two regular expressions R_1 and R_2, written R_1R_2 is also a regular expressions.

4. The closure (or iteration) of a regular expression R is written as R^* is also a regular expression.

5. If R is a Regular Expression then (R) is also a regular expression.

6. The Regular Expression over Σ are precisely those obtained by recursive applying rules 1 to 5 once or several times.

7. Nothing else is a regular expression.

Axioms:

Let R, S, T be regular expressions.

1. R|S = S|R
2. R|(S|T) = (R|S)|T
3. R(ST) = (RS)T
4. R(S|T) = RS | RT
5. εR = Rε = R (ε is identity for concatenation)

Identities for Regular Expressions:

Let a be a regular expression the,

1. $\phi|a = a$
2. $\phi a = a\phi = \phi$
3. $\varepsilon a = a\varepsilon = a$
4. $\varepsilon^* = \varepsilon$ and $\phi^* = \varepsilon$
5. $a|a = a$
6. $a^*a^* = a^*$
7. $aa^* = a^*a$
8. $(a^*)^* = a^*$
9. $\varepsilon|aa^* = a^* = \varepsilon|a^*a$
10. $(ab)^*a = a(ba)^*$
11. $(a|b)^* = (a^*b^*)^* = (a^*|b^*)^*$
12. $(a|b)c = ab|bc$ and $c(a|b) = ca|cb$

More Examples: (For C words)

digit = 0|1|.....|9

alpha = a|b|.....|z|A|B|.....|Z|_

sign = ε|+|-

integer = digit.digit*

sign_integer = sign.integer

identifier = alpha.(alpha|digit)*

string_constant = ".(alpha|digit)*."

FA from Regular Expression:

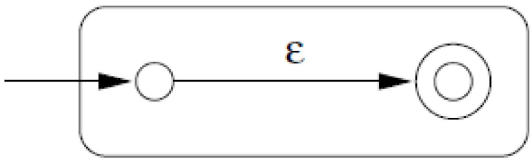
Theorem: For every regex R we can construct and ε-NFA A, such that L(A) = L(R).

Algorithm: Let E is a Regular Expression

1. Create a start and final state.
2. Try to recognize E.

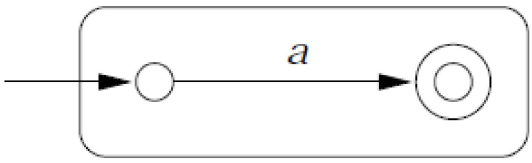
Case A:

a. E is ε, ϕ and a then,



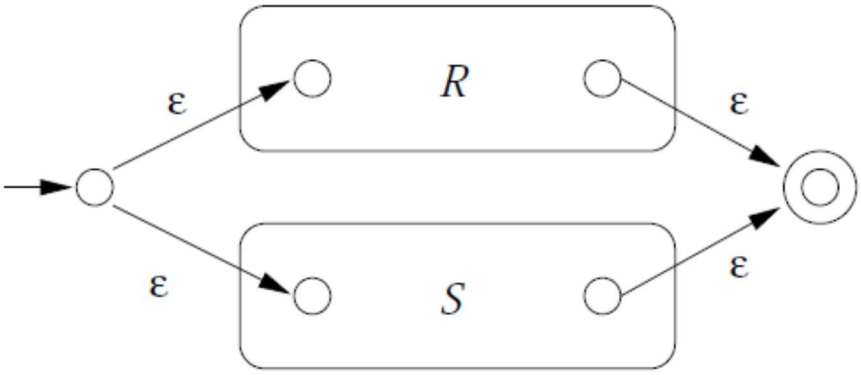
(a)

(b)

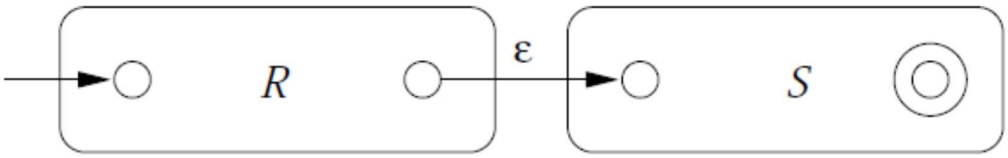


(c)

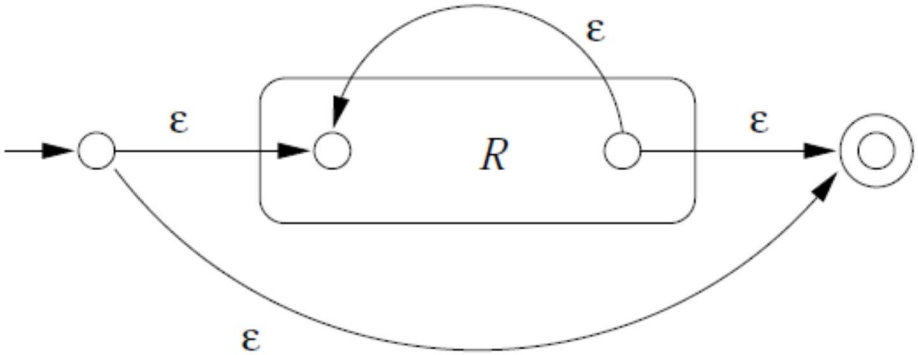
b. E is $R \mid S$, RS, and R^*



(a)



(b)

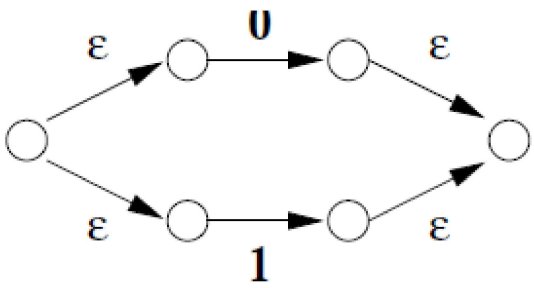


(c)

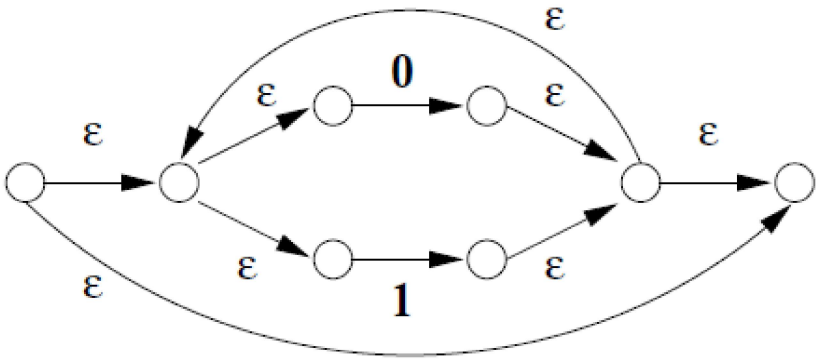
Class Work:

Create automata for $(0|1)^*1(0|1)$.

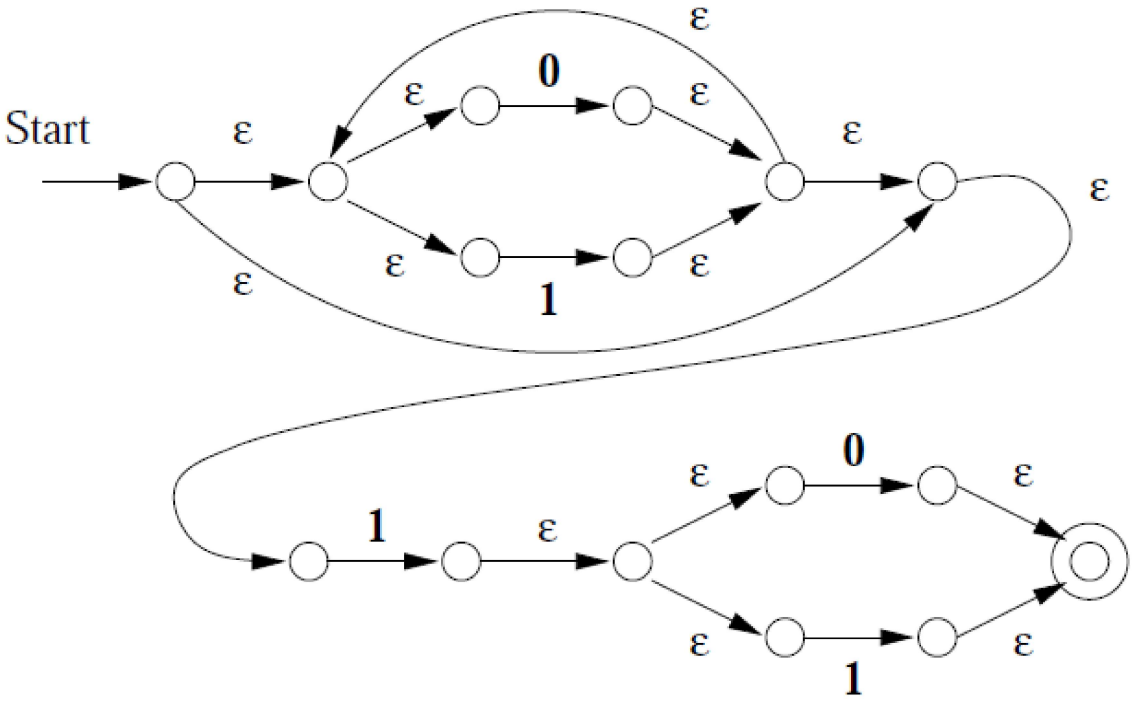
1.



2.



3.



2.4 Pumping Lemma for Regular Languages

Let L be regular. Then

$\exists n, \forall w \in L : |w| \geq n \Rightarrow w = xyz$

such that,

1. $y \neq \epsilon$
2. $|xy| \leq n$
3. $\forall k \geq 0, xy^k z \in L$

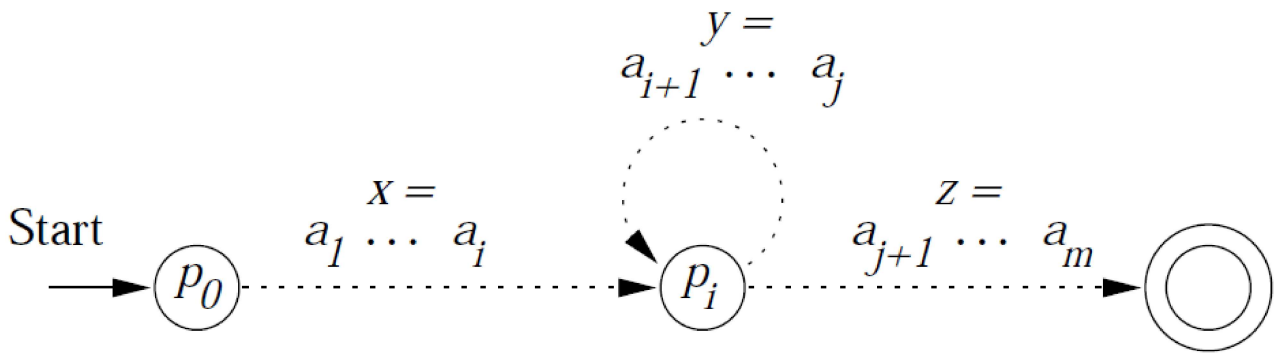
Proof:

Suppose L is regular. The L is recognized by some DFA A with, say, n states.

Let $w = a_1a_2 \ldots a_m \in L$, $m > n$.
Let $p_i = \delta\hat{}(q_0, a_1a_2 \ldots a_i)$.
$\Rightarrow \exists i < j : p_i = p_j$.

Now, w = xyz, where

1. $x = a_1a_2 \ldots a_i$
2. $y = a_{i+1}a_{i+2} \ldots a_j$
3. $z = a_{j+1}a_{j+2} \ldots a_m$



Evidently $xy^kz \in L$, for any $k \geq 0$.

Example - 1: Let L be the language of strings with equal number of zero's and one's.

Suppose L is regular. Then L = L(A), for some DFA A with, say, n states, and $w = 0^n1^n \in L(A)$.

By the pumping lemma w = xyz, |xy| ≤ n, $y \neq \epsilon$ and $xy^kz \in L(A)$

$$w = \underbrace{000\ldots}_{x}\underbrace{\ldots0}_{y}\underbrace{0111\ldots11}_{z}$$

In particular, $xz \in L(A)$, but xz has fewer 0's than 1's.

$\Rightarrow L(A) \neq L$.

Example – 2: Suppose L = {1^p: p is prime} were regular.

Then L = L(A), for some DFA A with, say n, states.

Choose a prime p ≥ n + 2.

$$w = \overbrace{\underbrace{111\cdots}_{x}\underbrace{\cdots\cdots}_{y}\underbrace{1}_{z}\underbrace{1111\cdots 11}_{z}}^{p}$$
$$|y|=m$$

Now $xy^{p-m}z \in L(A)$.

$$|xy^{p-m}z| = |xz| + (p - m)|y|$$
$$= (p - m) + (p - m)m$$
$$= (1 + m)(p - m)$$

which is not prime unless one of the factors is 1.

- $y \neq \epsilon \Rightarrow 1 + m > 1$
- $m = |y| \leq |xy| \leq n, \;\; p \geq n + 2$
  $\Rightarrow p - m \geq n + 2 - n = 2.$

⇉ $L(A) \neq L$

2.6 Decision Properties

We consider the following:

1. Converting among representations for regular languages.
    1. An algorithm must be guaranteed to halt after a finite number of steps.
    2. Each step of the algorithm must be well specified (deterministic rather than nondeterministic)

2. Is L(M) = ϕ?
    1. Simulate M on all strings of length between 0 and n-1 where M has n states.
    2. Output no if and only if all strings are rejected.
    3. Otherwise output yes.

3. Is w ∈ L(M)? Or is M accepts w ?
    a. Simply execute M on x.
    b. Output yes if we end up at an accepting state.

4. Do two descriptions define the same language? or is $L(M_1) = L(M_2)$?

a. Construct FA M_{1-2} from M_1 and M_2 which recognizes the language $L(>M_1) - L(>M_2)$.

b. Test if $L(M_{1-2})$ is empty.

c. Construct FA M_{2-1} from M_1 and M_2 which recognizes the language $L(>M_2) - L(>M_1)$.

d. Answer yes if and only if both answers were yes.